```
}

object SendProducer {
  def apply[K, V](settings: ProducerSettings[K, V])     ❶
       (implicit system: ClassicActorSystemProvider):   ❶
    SendProducer[K, V] = new SendProducer(settings, system.classicSystem)
  ...
}
```

❶ Curried factory

You can use it like this in its curried form.

```
val sendProducer = SendProducer(producerSettings)(system)
```

Now that you have the producer, you need a coordinated termination for it. You must close the producer before you exit the `ActorSystem` so the producer registry is freed. Otherwise, the attempt to restart the application will fail due to a conflicting registry. Kafka does not allow the same producer to be registered multiple times once it is open.

You can do this with `CoordinatedShutdown`, an extension that registers tasks—internal to the `ActorSystem` or from the user—to be executed when the system is shut down. There are several phases, but only three are intended to be used by Akka users:

- `Before-service-unbind`—The tasks set here run at the very beginning. This is the initial phase of the shutdown. You saw several examples of unbinding HTTP services in chapter 11.
- `Before-cluster-shutdown`—The tasks set here run after the service shutdown and before the cluster initiates the shutdown.